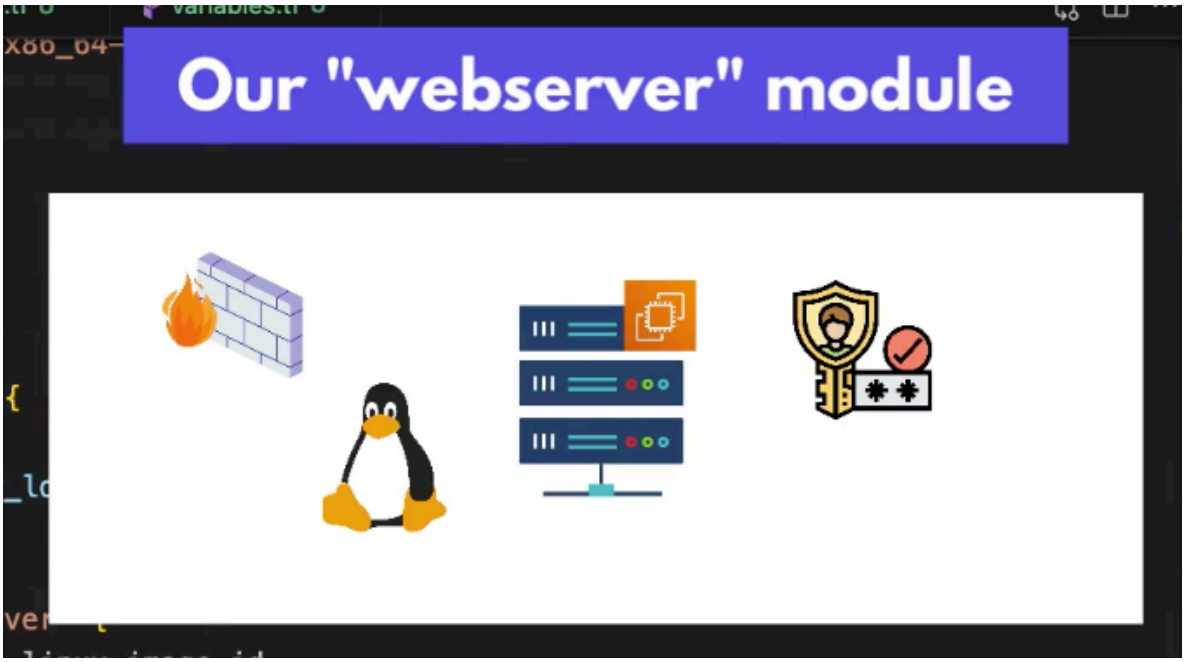
Module 12 - chapter 18
Modules in Terraform - Part 3

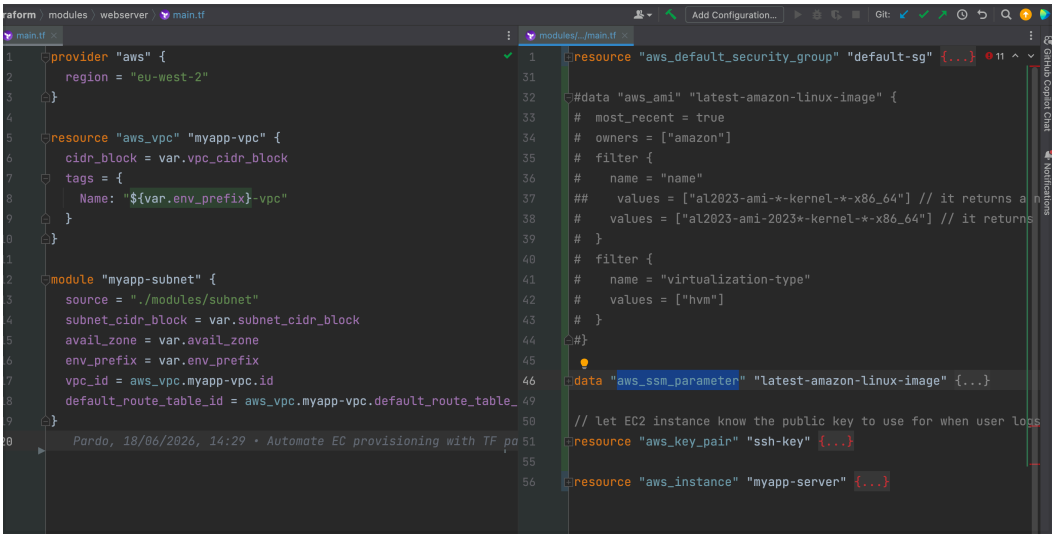
The final modularized project files used in this lecture can be found here: https://gitlab.com/twn-devops-bootcamp/latest/12-terraform/terraform-learn/-/tree/feature/modules

Now let's configure the web server module:

- aws_instance
- aws_key_pair
- aws_ssm_parameter -> data (nana uses 'aws_ami')
- aws_deault_security_group

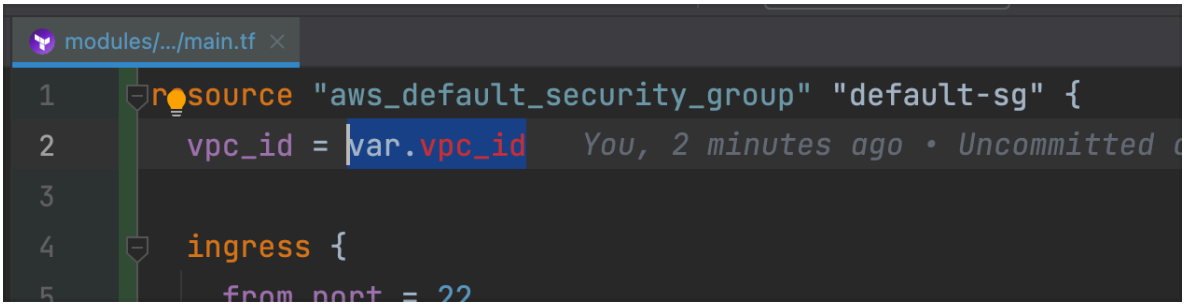


So we need to move 3 resources and one data:

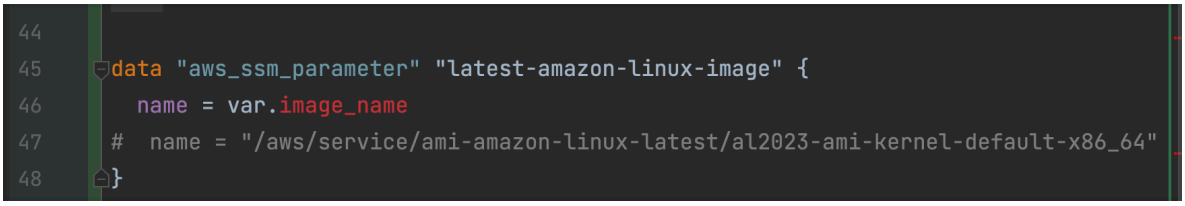


So now I refactor the code to use variables for those external references in web server/main.tf that used to be internal references in root main.tf

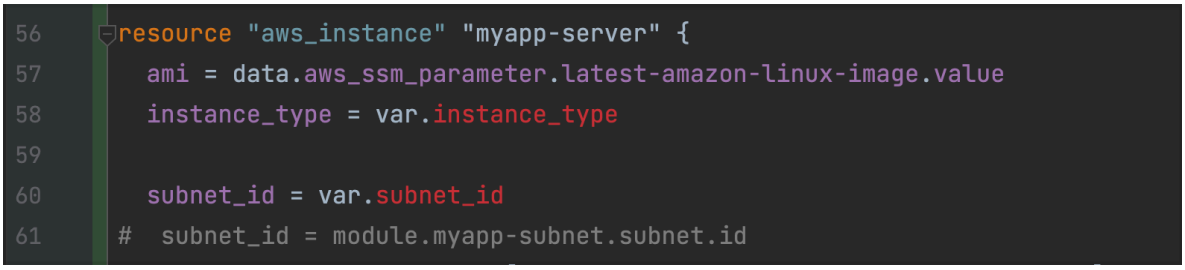
See line 2



Also we can use a variable instead of a hardcoded image name

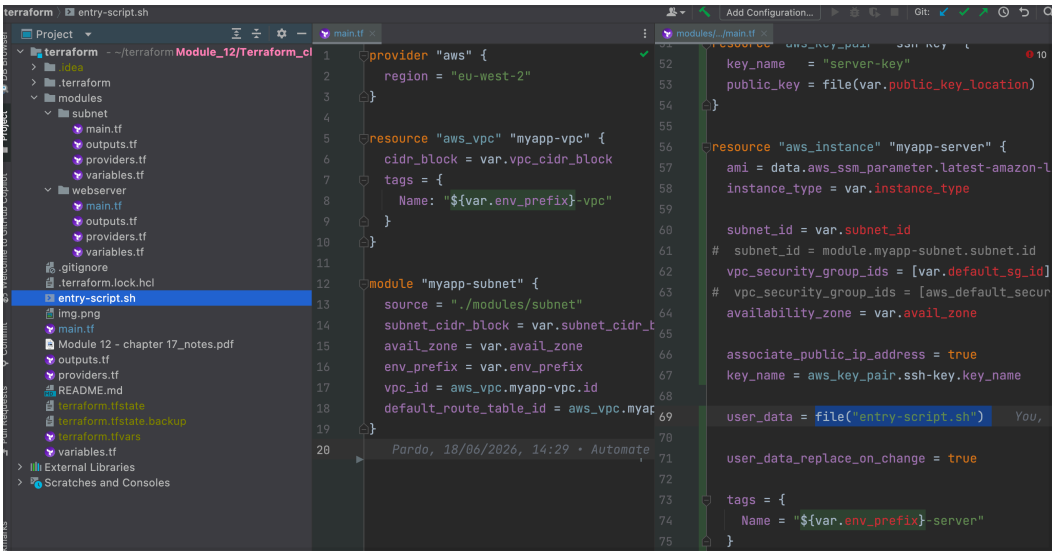


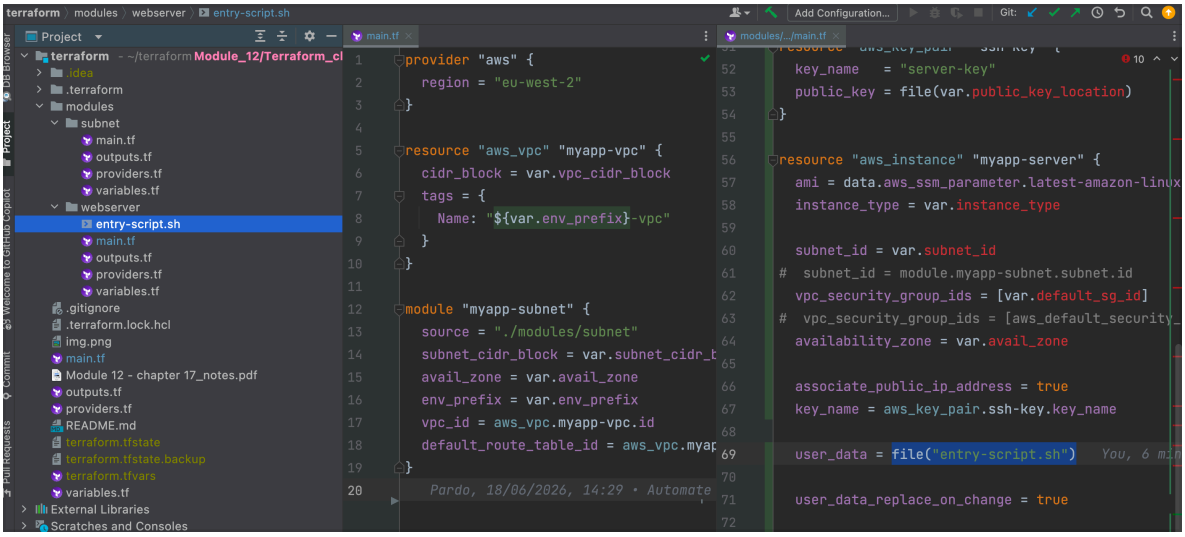
We move the aws_instance code to a submodule so now the aws_insatnce doesn't have access to the 'module' resource named myapp-subnet in root main.tf so we need to replace it why a variable as well see line 60 👇



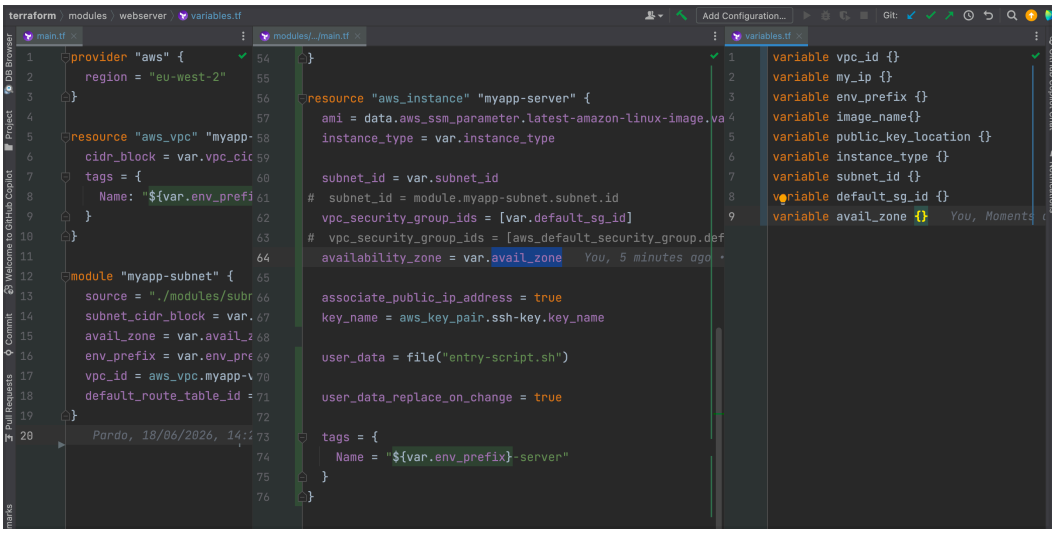
And move the script file from the root folder to the webserver submodule, so

line 69 in webserver/main.tf can find it 👇

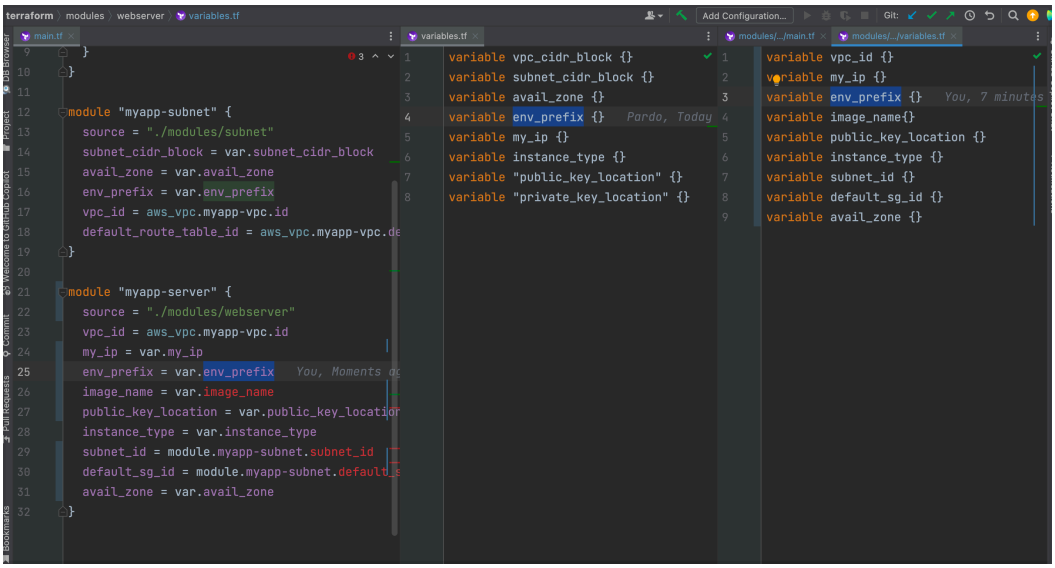




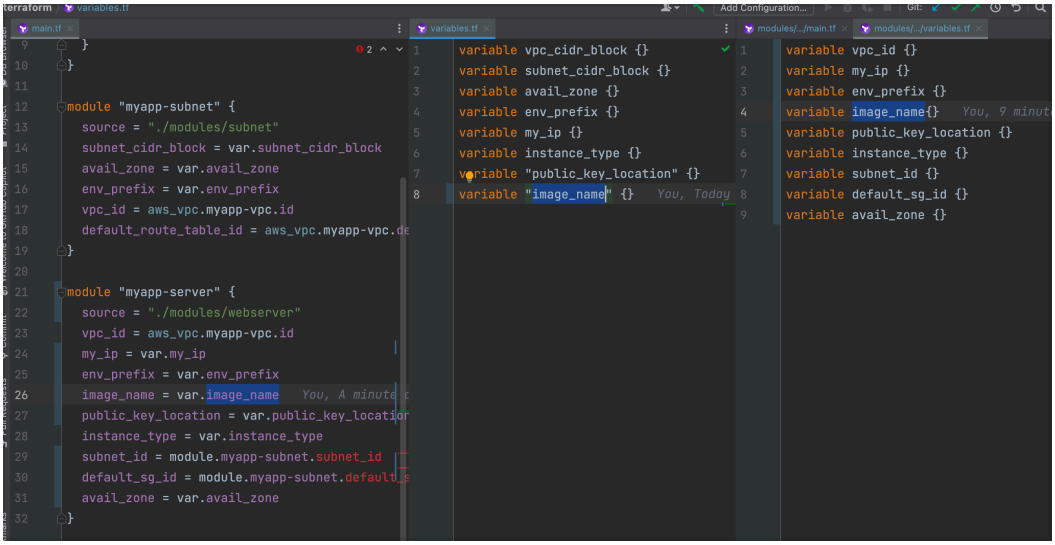
Now let's declare the variables in the webserver/variables.tf file



Now we can reference the module in the root main.tf file

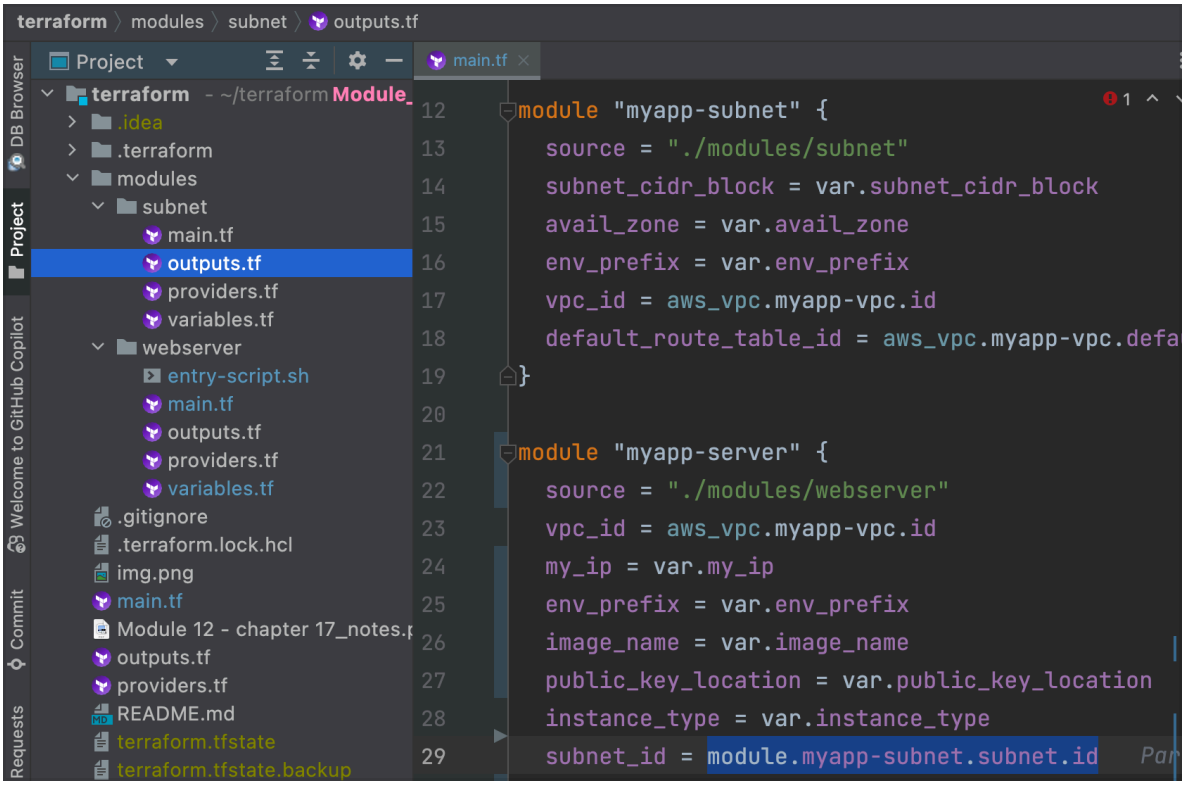


I add ‘image_name’ to the root variables.tf file (and get rid of the private key var as this was used with provisioners in chapter 15 but we don’t need it anymore)

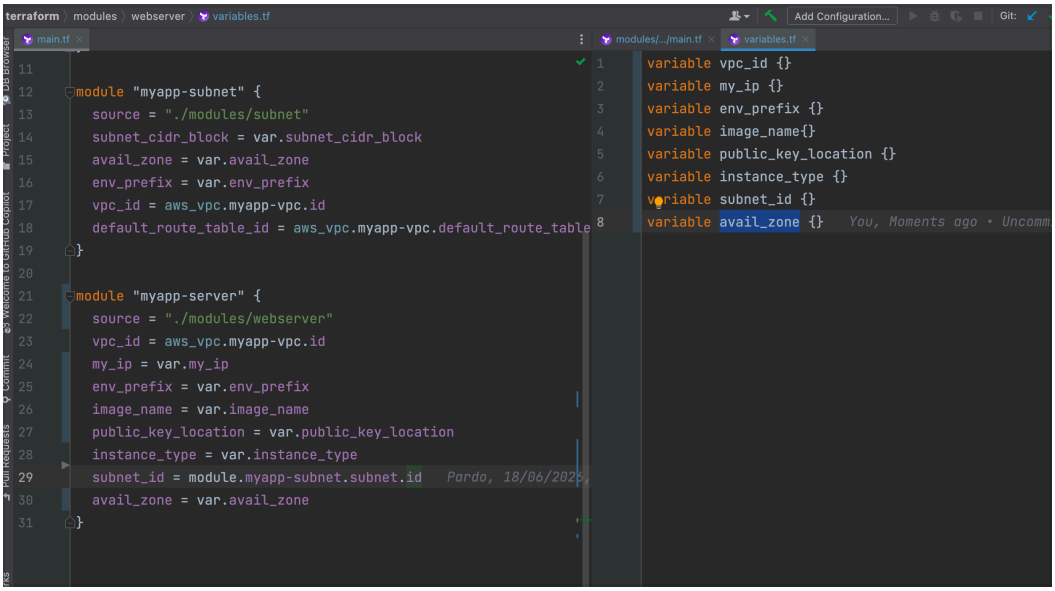


For the subnet_id attribute we can reference the module “myapp-subnet” because it is created in the same file and then we get the output called ‘subnet’ to get the object and then we get its id

-> module.myapp-subnet.subnet.id

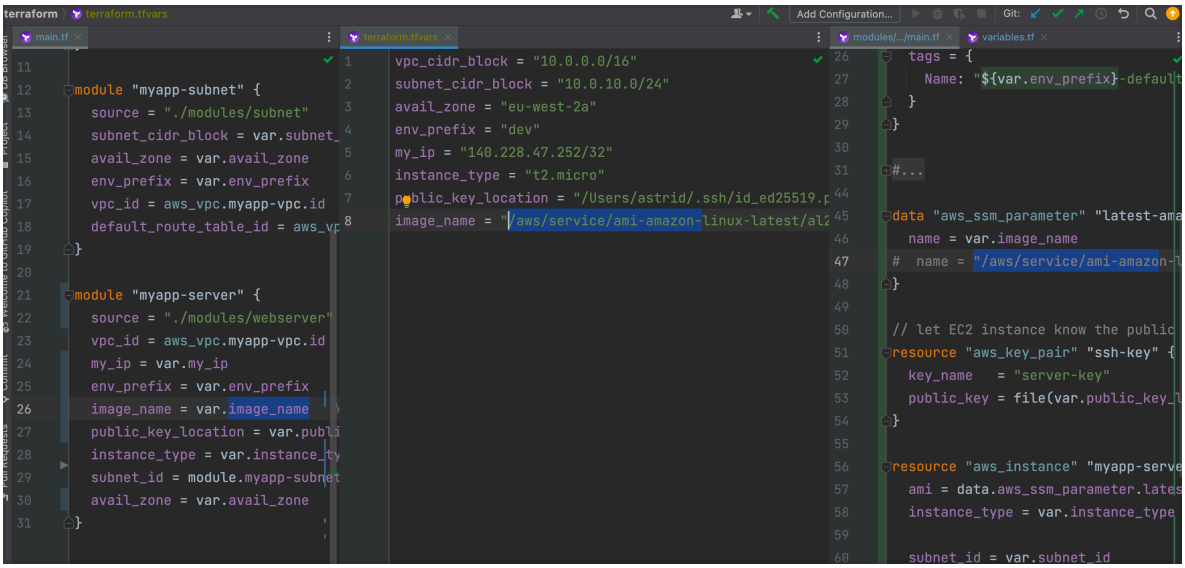


And this is the result (see we don't have 'default_sg_id' var any longer because we didn't need to replace the reference with a variable in the first place)

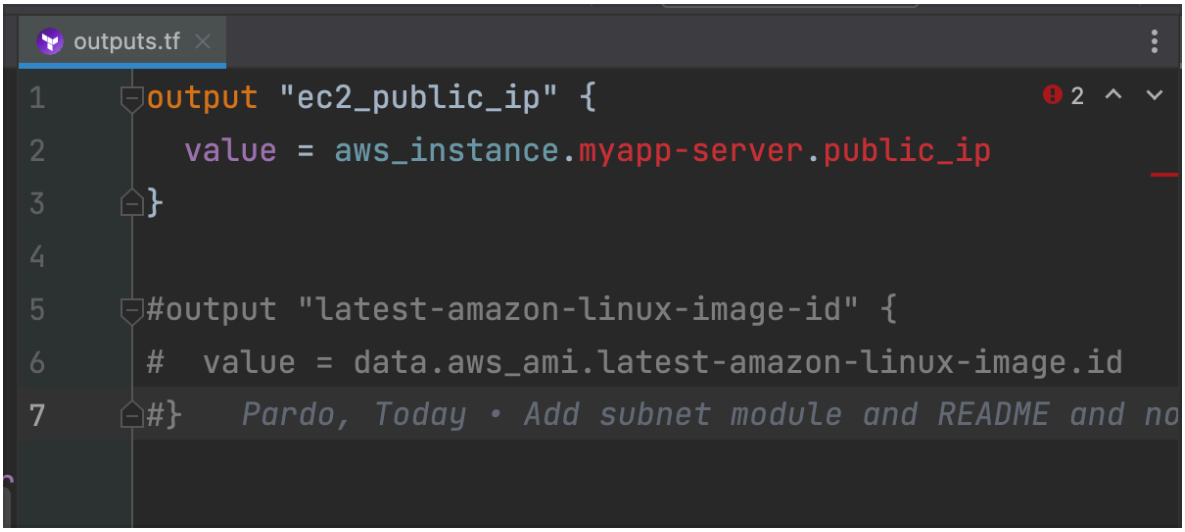


Now let's make sure the values of the variables are set in the terraform.tfvars

We need to add image_name



Now we need to review the root outputs,tf file

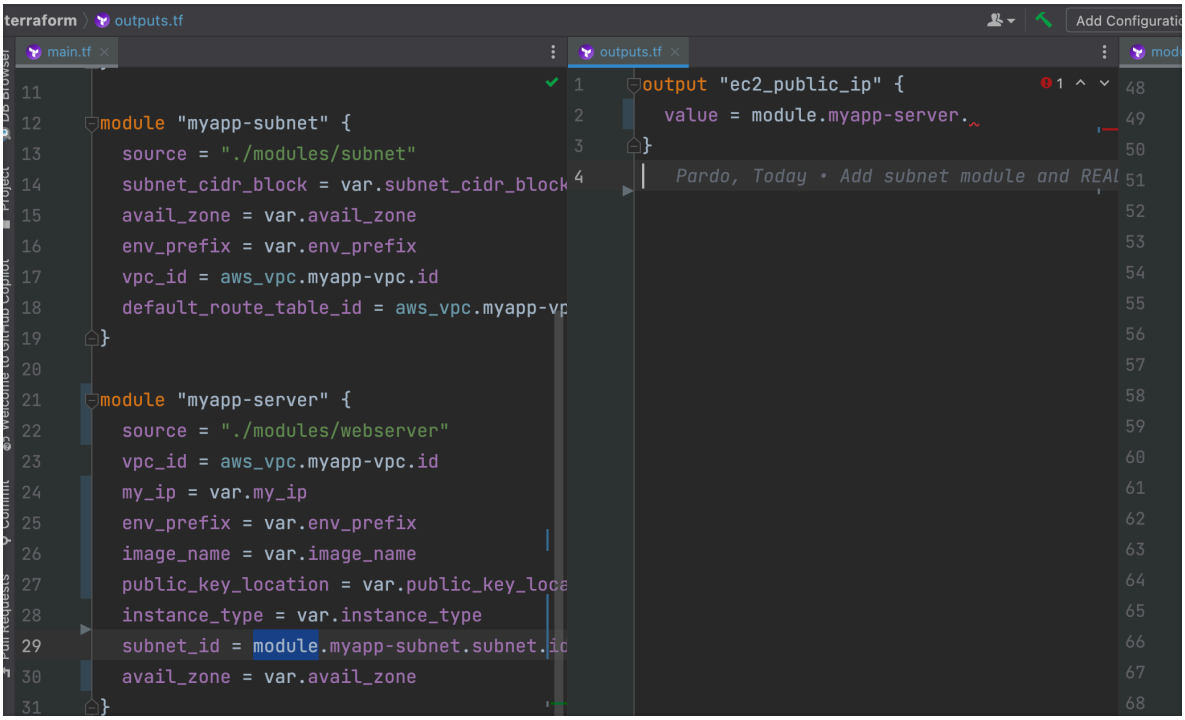


I have one output -> Nana uses two outputs but I use'aws_ssm_parameter' so I can't get an output from data.

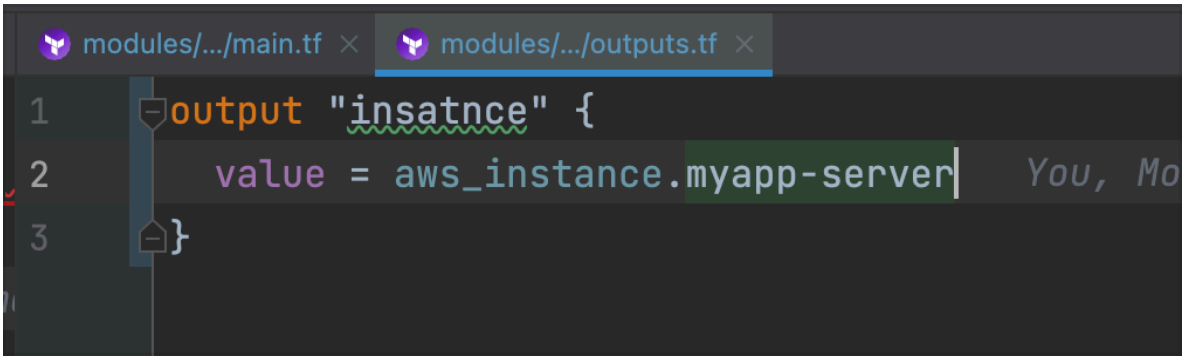
The outputs don't work any more because the resources that returns them have been moved to submodules.

So I need to update the way we reference the output, with this syntax
-> module.<module_name>.<output_name>.<object_attribute>

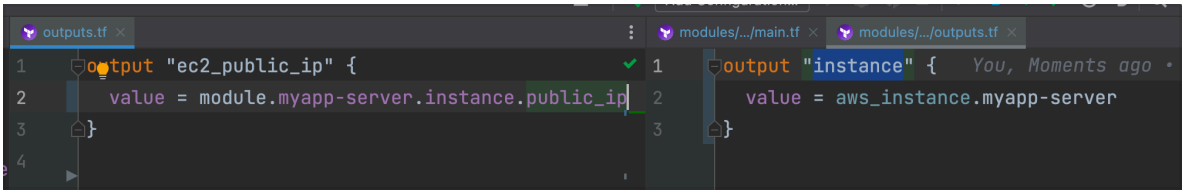
Below we have module.<module_name> (see line 2 root outputs.tf 👇)



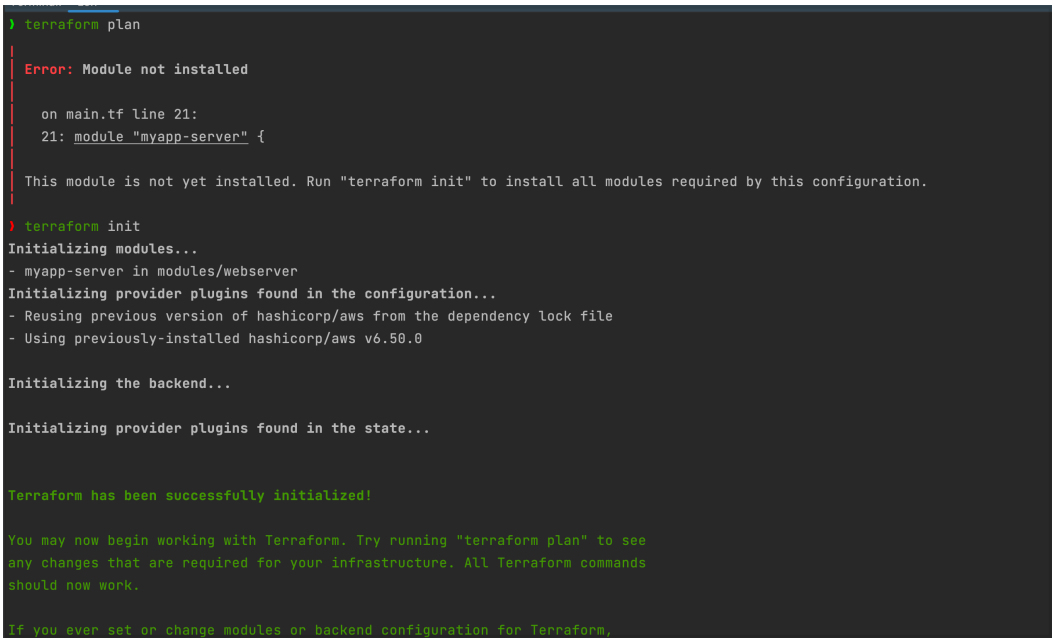
We need to create the output of the werserver/main.tf in the werserver/ outputs.tf and we decided to output the whole instance object



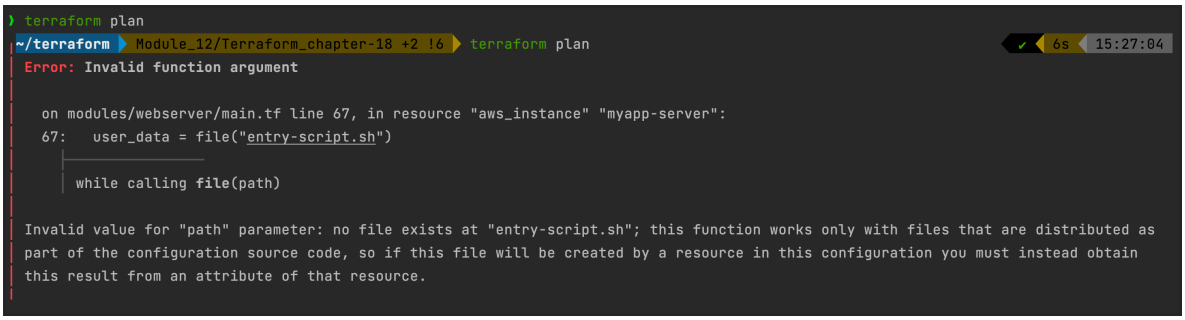
And now we can complete the reference
-> module.myapp-server.instance.public_ip



Now we execute
-> terraform init

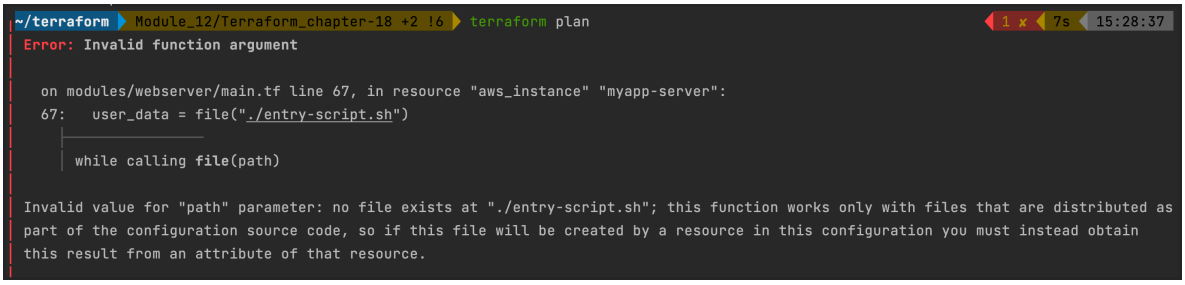


And now we can check the configuration with
-> terraform plan



Path error 👆

Then I tried
-> ./entry-script.sh



Path error 👆 again

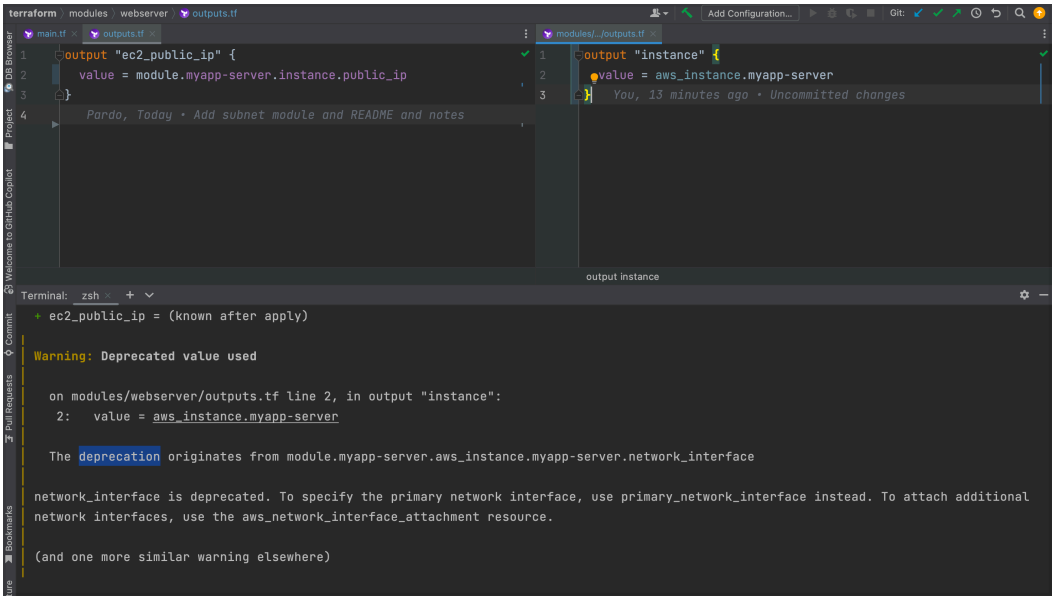
Problem:
The ./ in file() resolves relative to where you run terraform plan (the root), not relative to the module file. So it can't find entry-script.sh inside modules/webserver/.

Fix — use path.module, which always points to the module's own directory:

```
user_data = file("${path.module}/entry-script.sh")
```

Make sure entry-script.sh is sitting next to modules/webserver/main.tf.

Now I get this warning

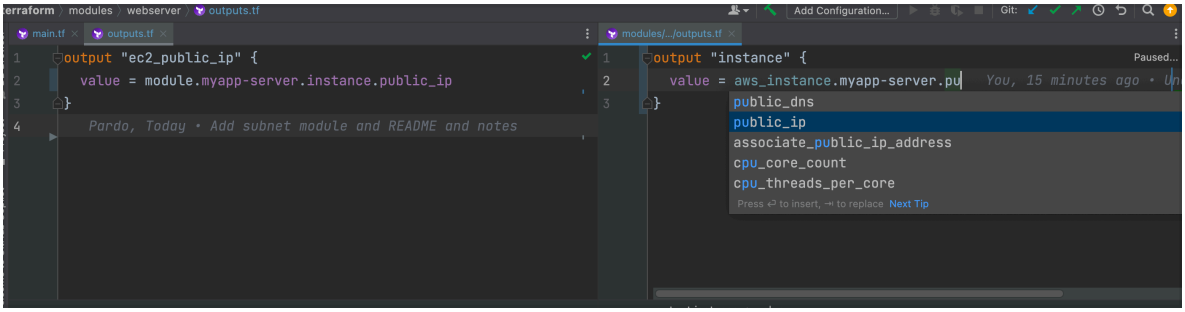


Problem
The warning is because you're outputting the entire aws_instance resource, which includes the deprecated network_interface attribute.

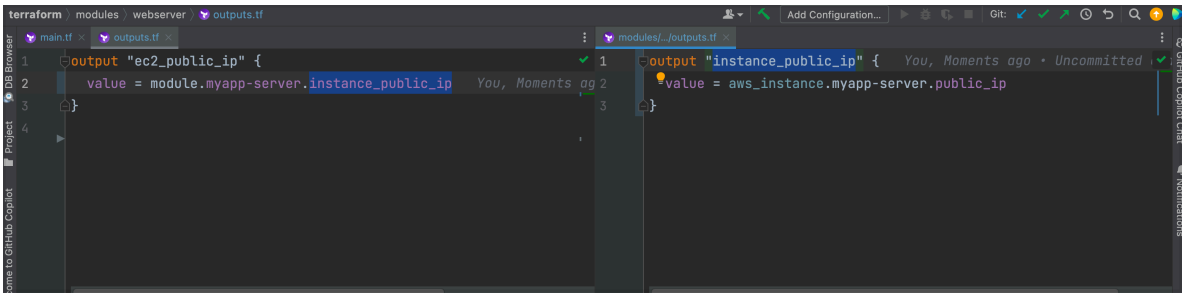
Fix
Since you only need public_ip, output just that attribute instead of the whole instance.

```
modules/webserver/outputs.tf:
output "instance_public_ip" {
  value = aws_instance.myapp-server.public_ip
}

Root outputs.tf:
output "ec2_public_ip" {
  value = module.myapp-server.instance_public_ip
}
```

This removes the deprecated attribute from the output entirely and is cleaner — only expose what you actually need from a module.



And let’s rename the output from ‘instance’ to ‘instance_public_ip’ as it is more descriptive



-> terraform plan

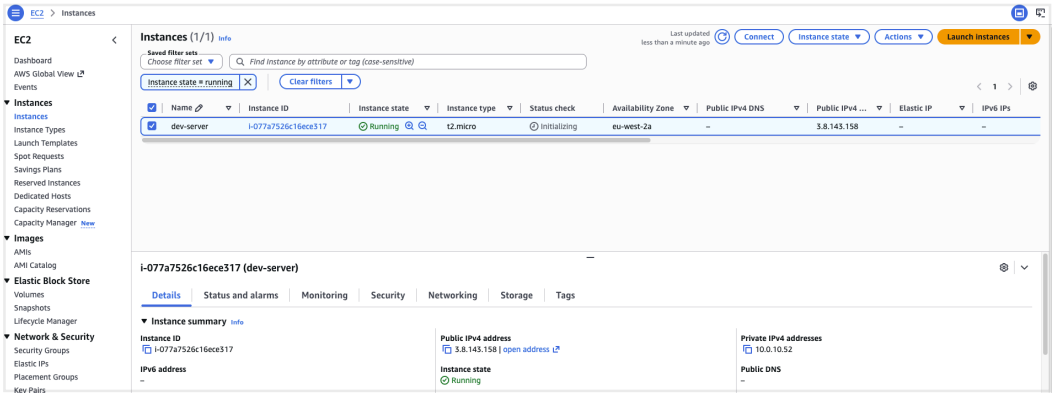
And no errors

```
          + "Name" = "dev-subnet-1"
        }
      + vpc_id                                     = (known after apply)
    }

Plan: 7 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + ec2_public_ip = (known after apply)

─────────────────────────────────────────────────────────────────────────────

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly th
apply" now.

~/terraform  Module_12/Terraform_chapter-18 +2 !6
```

-> terraform apply

```
Changes to Outputs:
  + ec2_public_ip = (known after apply)
module.myapp-server.aws_key_pair.ssh-key: Creating...
aws_vpc.myapp-vpc: Creating...
module.myapp-server.aws_key_pair.ssh-key: Creation complete after 0s [id=server-key]
aws_vpc.myapp-vpc: Creation complete after 1s [id=vpc-07fa3069a0c6606d7]
module.myapp-subnet.aws_internet_gateway.myapp-igw: Creating...
module.myapp-subnet.aws_subnet.myapp-subnet-1: Creating...
module.myapp-server.aws_default_security_group.default-sg: Creating...
module.myapp-subnet.aws_internet_gateway.myapp-igw: Creation complete after 1s [id=igw-09ca72f667b49d0c1]
module.myapp-subnet.aws_default_route_table.main-rtb: Creating...
module.myapp-subnet.aws_subnet.myapp-subnet-1: Creation complete after 1s [id=subnet-0a855b8c25c45650c]
module.myapp-subnet.aws_default_route_table.main-rtb: Creation complete after 0s [id=rtb-0ddc3ce1193d09b15]
module.myapp-server.aws_default_security_group.default-sg: Creation complete after 3s [id=sg-0c047838dd28ef732]
module.myapp-server.aws_instance.myapp-server: Creating...
module.myapp-server.aws_instance.myapp-server: Still creating... [00m10s elapsed]
module.myapp-server.aws_instance.myapp-server: Still creating... [00m20s elapsed]
module.myapp-server.aws_instance.myapp-server: Creation complete after 22s [id=i-077a7526c16ece317]

Apply complete! Resources: 7 added, 0 changed, 0 destroyed.

Outputs:

ec2_public_ip = "3.8.143.158"

~/terraform  Module_12/Terraform_chapter-18 +2 !6                                  ✓  35s
```

And we check the AWS UI



And it works 🥳

Let’s ssh into the EC2 instance and check if the script file run properly and installed docker

```
❯ ssh ec2-user@3.8.143.158

The authenticity of host '3.8.143.158 (3.8.143.158)' can't be established.
ED25519 key fingerprint is SHA256:GD7+6eAoJJwk770XskHJKxKCzh//n0/pQ/UMIUDx/tw.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '3.8.143.158' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-0-10-52 ~]$ docker -v
Docker version 25.0.14, build 0bab007
[ec2-user@ip-10-0-10-52 ~]$
```

Wrap up